

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete functional and non-functional requirements for the AI-Powered HR Recruitment System. The system uses machine learning to automate resume screening and predict the most suitable job role for a given candidate based on their resume content.

The document is intended for software developers, data scientists, QA engineers, HR stakeholders, and academic reviewers.

1.2 Project Scope

The AI-Powered HR Recruitment System is an NLP-based multi-class classification application that:

- Accepts candidate resumes in PDF, DOCX, or image (JPG/PNG) format
- Extracts and cleans resume text using preprocessing pipelines
- Classifies the resume into one of 51 job roles using trained ML models
- Computes cosine similarity between resume and job description vectors
- Outputs a ranked list of best-match job roles with confidence scores
- Provides a Streamlit-based web interface for HR personnel

1.3 Definitions, Acronyms & Abbreviations

Term / Acronym	Definition
NLP	Natural Language Processing — AI field for text analysis
TF-IDF	Term Frequency-Inverse Document Frequency — text vectorization technique
ML	Machine Learning — subset of AI for pattern learning
SRS	Software Requirements Specification
OCR	Optical Character Recognition — extracting text from images
API	Application Programming Interface
UI	User Interface
HR	Human Resources
JD	Job Description
CV	Curriculum Vitae / Resume

1.4 References

- Scikit-learn Documentation: <https://scikit-learn.org>
- Streamlit Framework: <https://streamlit.io>
- PyMuPDF (fitz) for PDF parsing: <https://pymupdf.readthedocs.io>
- XGBoost Documentation: <https://xgboost.readthedocs.io>
- LightGBM Documentation: <https://lightgbm.readthedocs.io>

2. Overall System Description

2.1 Product Perspective

The HR Recruitment System is a standalone desktop/web application built on Python. It interfaces with the file system to read uploaded resumes and uses trained ML models stored as serialized pickle files. The Streamlit frontend communicates directly with the ML backend — no external API or database server is required.

2.2 Product Functions

The core functions of the system are:

1. Resume Ingestion: Accept and parse PDF, DOCX, PNG, JPG files
2. Text Preprocessing: Lowercase, remove special characters, remove stopwords
3. Feature Extraction: TF-IDF vectorization with unigrams and bigrams
4. Job Role Classification: Predict one of 51 job roles using trained ML model
5. Cosine Similarity Ranking: Score resume against job description vectors
6. Result Display: Ranked output with match percentage and HR recommendation

2.3 User Classes & Characteristics

User Class	Skill Level	Responsibilities
HR Manager	Non-technical	Upload resumes, view match reports, make hiring decisions
Data Scientist	Expert	Train models, tune hyperparameters, evaluate performance
System Admin	Technical	Deploy system, manage model files, maintain server
Job Applicant	End User	Submit resume for evaluation (indirect user)

2.4 Operating Environment

- OS: Windows 10/11, Ubuntu 20.04+, macOS 12+
- Python: Version 3.8 or above
- Browser: Chrome, Firefox, Edge (for Streamlit UI)
- RAM: Minimum 4 GB, Recommended 8 GB
- Storage: Minimum 2 GB free space
- CPU: Intel i5 / AMD Ryzen 5 or better

2.5 Constraints

- System works only with English-language resumes
- Maximum resume file size: 10 MB
- Supported image formats for OCR: JPG, JPEG, PNG only
- Model predictions are limited to 51 pre-trained job role categories
- No real-time training — model must be retrained offline for new data

3. Functional Requirements

FR-01: Resume File Upload

Requirement ID	FR-01
Title	Resume File Upload
Description	The system shall allow HR users to upload resume files in PDF, DOCX, PNG, or JPEG format through the web interface.
Input	Resume file (PDF / DOCX / PNG / JPEG)
Output	Extracted raw text from the resume
Priority	HIGH

FR-02: Text Extraction

Requirement ID	FR-02
Title	Multi-format Text Extraction
Description	The system shall extract text from PDF using PyMuPDF, from DOCX using python-docx, and from images using Tesseract OCR.
Input	Uploaded file object
Output	Plain text string of resume content
Priority	HIGH

FR-03: Text Preprocessing

Requirement ID	FR-03
Title	NLP Text Preprocessing Pipeline
Description	The system shall clean resume text by converting to lowercase, removing special characters and numbers, and filtering out predefined stopwords.
Input	Raw resume text
Output	Cleaned, normalized text string
Priority	HIGH

FR-04: Job Role Classification

Requirement ID	FR-04
Title	Multi-Class Job Role Prediction
Description	The system shall classify the candidate's resume into one of 51 job role categories using the best-performing trained ML model (Logistic Regression / SVM / Random Forest / XGBoost / LightGBM).

Input	TF-IDF vector of cleaned resume
Output	Predicted job role label with confidence score
Priority	HIGH

FR-05: Cosine Similarity Matching

Requirement ID	FR-05
Title	Job-Resume Cosine Similarity Score
Description	The system shall compute cosine similarity between the resume TF-IDF vector and predefined job description vectors for 4 target tech roles (Data Scientist, ML Engineer, Data Analyst, Software Engineer) and return a ranked match percentage.
Input	Resume vector + Job description vectors
Output	Ranked list of jobs with match % (0-100)
Priority	HIGH

FR-06: HR Decision Recommendation

Requirement ID	FR-06
Title	HR Recommendation with Threshold
Description	The system shall flag a candidate as 'Recommended' if their cosine similarity score for any job role is $\geq 60\%$, and 'Not Recommended' otherwise.
Input	Cosine similarity scores
Output	HR Decision: Recommended / Not Recommended
Priority	MEDIUM

FR-07: Batch Resume Screening

Requirement ID	FR-07
Title	Batch Upload Processing
Description	The system shall support batch upload of multiple resume files and process each independently, printing individual HR matching reports.
Input	Multiple resume files (uploaded via Google Colab or UI)
Output	Individual HR reports for each candidate
Priority	MEDIUM

4. Non-Functional Requirements

4.1 Performance Requirements

- Single resume classification must complete within 3 seconds
- Batch processing of 50 resumes must complete within 2 minutes
- Model loading from pickle file must complete within 5 seconds
- TF-IDF vectorization of a resume must complete within 1 second

4.2 Security Requirements

- Resume files must not be stored permanently on the server
- No personally identifiable information (PII) should be logged
- Model files (.pkl) must be protected from unauthorized modification
- Streamlit app should be served over HTTPS in production deployment

4.3 Reliability Requirements

- System must handle corrupted or empty file uploads gracefully with error messages
- System must maintain 99% uptime during business hours
- Error recovery: failed predictions must be logged without crashing the system

4.4 Usability Requirements

- HR users must be able to upload and receive results without technical training
- Match percentage must be displayed visually using progress bars
- Results page must clearly highlight 'Recommended' vs 'Not Recommended' status

4.5 Maintainability Requirements

- Model can be retrained and replaced by updating model.pkl without code changes
- Job descriptions in the HR database can be updated without model retraining
- All ML components are modular and independently testable

5. System Architecture & Design

5.1 High-Level Architecture

The system follows a 3-tier layered architecture:

Layer	Components	Description
Presentation	Streamlit UI	Web interface for HR users — file upload, result display, progress bars
Business Logic	ML Pipeline, NLP Engine, Similarity Engine	Core processing: text extraction, cleaning, vectorization, classification, matching
Data Layer	CSV Dataset, Pickle Models, Job DB	Training data, serialized models, and job description knowledge base

5.2 Component Description

5.2.1 Resume Parser Module

Handles multi-format resume ingestion. Dispatches PDF to PyMuPDF, DOCX to python-docx, and image files to Tesseract OCR. Returns raw text string.

5.2.2 Text Preprocessing Module

Implements the NLP preprocessing pipeline: lowercasing, regex-based noise removal, and stopword filtering using a custom stopword set. Outputs clean token sequence.

5.2.3 TF-IDF Vectorizer

Trained on 10,000 resume records with max_features=3000, ngram_range=(1,2). Converts cleaned resume text into sparse numeric feature vectors. Saved as tfidf.pkl.

5.2.4 Classification Engine

Loads the best-performing serialized model (model.pkl). Accepts TF-IDF vector as input and predicts one of 51 job role labels. Uses LabelEncoder (label_encoder.pkl) to decode predictions.

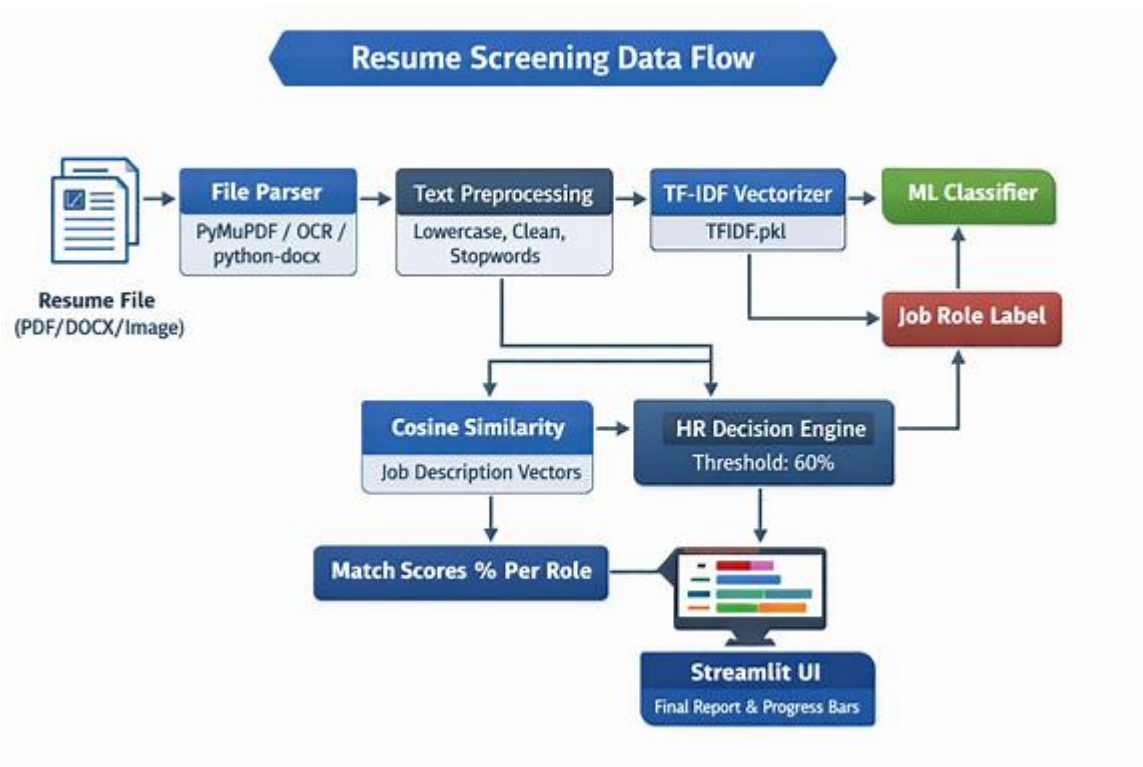
5.2.5 Similarity Engine

Maintains a job description knowledge base with 4 target roles. Computes cosine similarity between resume vector and each job vector. Returns ranked match scores.

5.2.6 HR Decision Engine

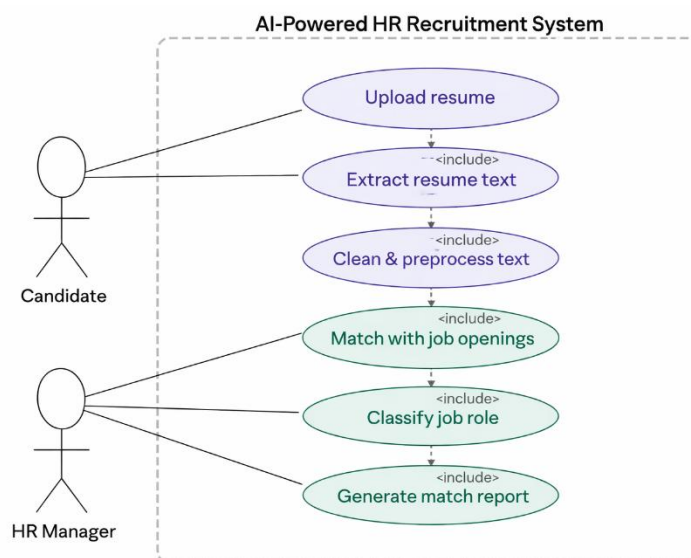
Applies a 60% threshold to cosine similarity scores. Generates structured HR recommendations including best-match role, match percentage, and accept/reject decision.

5.3 Data Flow Diagram

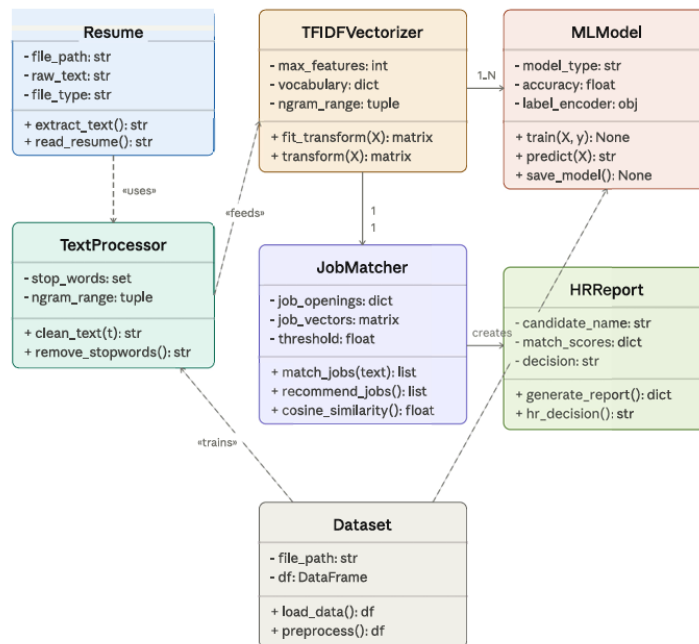


6. UML & ER Diagrams

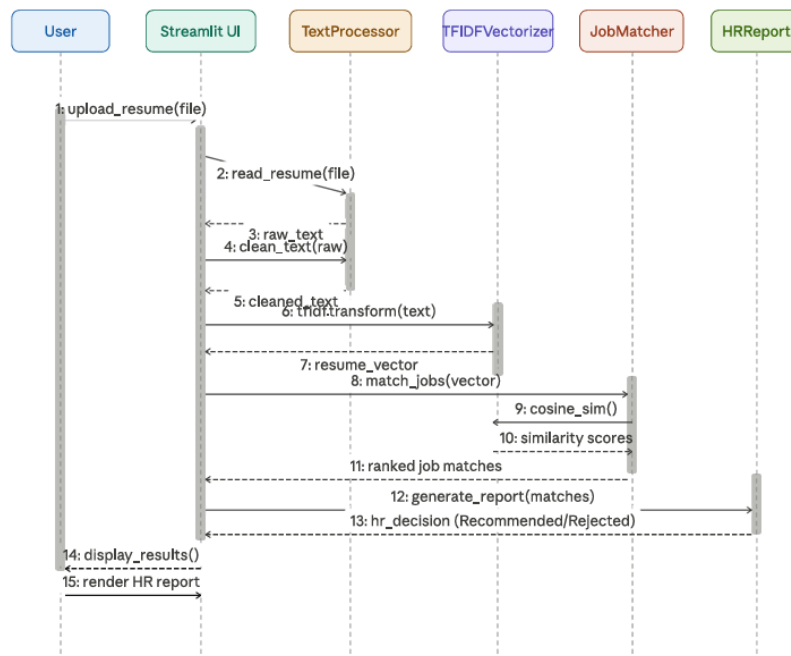
6.1 Use Case Diagram — Actors & Use Cases



6.2 Class Diagram — Core Classes

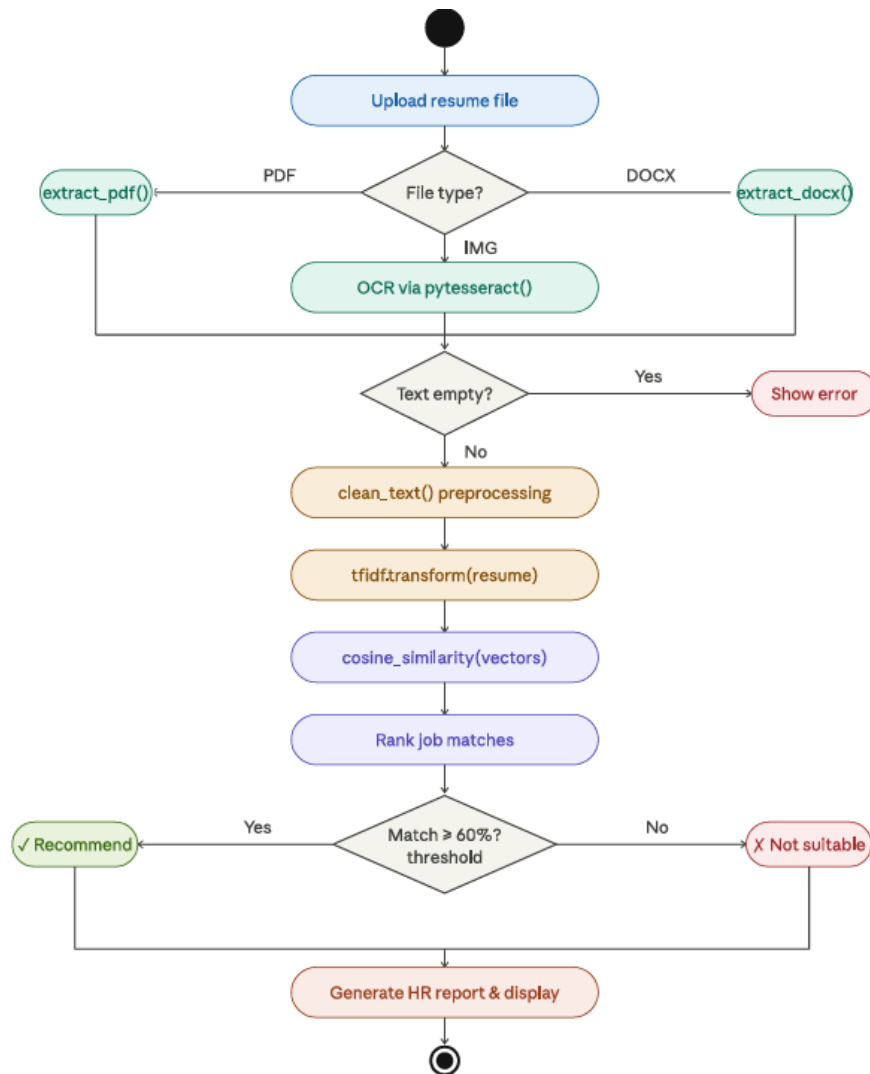


6.3 Sequence Diagram — Resume Screening Flow



6.4 Activity Diagram — System Processing Flow

Start → Upload File → [File Empty? → Error] → Extract Text → Clean Text → Vectorize → Classify Role → Compute Similarity → [Score >= 60%? → Recommended | Score < 60% → Not Recommended] → Display Report → End



7. Database Design

7.1 Dataset Schema (job_applicant_dataset.csv)

The primary data source is a CSV file with 10,000 rows. Below is the logical schema:

Column Name	Data Type	Constraints	Description
Job Applicant Name	VARCHAR(100)	NOT NULL	Full name of the applicant
Age	INTEGER	NOT NULL, 25–55	Age of the applicant
Gender	VARCHAR(10)	NOT NULL	Male or Female
Race	VARCHAR(30)	NOT NULL	Mongoloid/Asian, White/Caucasian, Negroid/Black
Ethnicity	VARCHAR(30)	NOT NULL	One of 21 ethnic groups
Resume	TEXT	NOT NULL	Free-text resume with skills, degree, certifications

Column Name	Data Type	Constraints	Description
Job Roles	VARCHAR(50)	NOT NULL, FK → JobRoles	Target label — one of 51 job roles (PRIMARY TARGET)
Job Description	TEXT	NOT NULL	Full job description for the target role
Best Match	INTEGER	0 or 1	Binary label — 1 if best match, 0 otherwise

7.2 Model Artifact Storage

File Name	Format	Contents
model.pkl	Pickle (.pkl)	Best trained ML classifier (LR / RF / XGB / LGBM / SVM)
tfidf.pkl	Pickle (.pkl)	Fitted TF-IDF vectorizer (vocab, IDF weights)
label_encoder.pkl	Joblib (.pkl)	LabelEncoder mapping class indices to 51 job role names

7.3 ER Diagram — Entity Relationships

Entity	Cardinality	Related Entity	Relationship
JobApplicant	Many	JobRole	An applicant is classified into ONE job role
JobRole	One	JobDescription	Each job role has ONE job description
JobApplicant	One	Resume	Each applicant has ONE resume
MLModel	One	TFIDFVectorizer	Model depends on ONE fitted vectorizer
MLModel	One	LabelEncoder	Model uses ONE encoder for 51 class labels

8. Installation & User Guide

8.1 Prerequisites

- Python 3.8+ installed
- pip package manager
- Google Chrome or Firefox browser
- Tesseract OCR engine (for image resumes)

8.2 Installation Steps

Step 1: Clone/Download Project

Download the project files to your local machine. Ensure the following files are present: MLProject[Notebook_.ipynb], job_applicant_dataset.csv, and the app directory with Streamlit code.

Step 2: Install Python Dependencies

```
pip install pymupdf python-docx pytesseract scikit-learn shap matplotlib seaborn
xgboost lightgbm streamlit
```

Step 3: Install Tesseract OCR

- Windows: Download from <https://github.com/UB-Mannheim/tesseract/wiki>
- Ubuntu/Debian: `sudo apt-get install tesseract-ocr`
- macOS: `brew install tesseract`

Step 4: Train & Save Models (Jupyter Notebook)

Open ML_Project_Notebook_.ipynb in Jupyter or Google Colab. Run all cells sequentially. Upon completion, model.pkl, tfidf.pkl, and label_encoder.pkl will be saved to the working directory.

Step 5: Launch Streamlit App

```
streamlit run app.py
```

8.3 User Guide — Using the HR System

Step	Action	Details
1	Open Browser	Navigate to http://localhost:8501 after launching Streamlit
2	Upload Resume	Click 'Upload Resume' button. Select PDF, DOCX, PNG, or JPG file.
3	View HR Report	System automatically processes and displays match scores
4	Check Recommendation	Green box = Recommended ($\geq 60\%$), Red box = Not Recommended
5	Review Job Matches	Progress bars show cosine similarity % for each job role
6	Final Decision	HR Manager makes hiring decision based on report

8.4 Troubleshooting

Error / Issue	Solution
ModuleNotFoundError	Run pip install command in Step 2 again
Tesseract not found	Set tesseract path: <code>pytesseract.pytesseract.tesseract_cmd = r'C:/...path...'</code>
Empty resume extraction	Ensure file is not corrupted. Try converting PDF to a newer version.

Error / Issue	Solution
Low match scores	Resume may lack technical keywords. Expand job description database.
Streamlit not launching	Check Python version (≥ 3.8). Reinstall streamlit.

9. Test Cases

TC ID	Test Description	Input	Expected Output	Status
TC-01	Valid PDF resume upload	Sample PDF resume file	Text extracted successfully	PASS
TC-02	Valid DOCX resume upload	Sample DOCX resume	Text extracted successfully	PASS
TC-03	Image resume (PNG) OCR	Resume as PNG image	OCR text extracted	PASS
TC-04	Empty file upload	Empty PDF file	Error: Empty Resume	PASS
TC-05	Data Scientist resume	Python ML TensorFlow NLP skills	Data Scientist predicted	PASS
TC-06	Software Engineer resume	Java C++ REST API Git skills	Software Engineer predicted	PASS
TC-07	High cosine similarity	Strong ML skill resume	Score $\geq 60\%$ → Recommended	PASS
TC-08	Low cosine similarity	Unrelated skills resume	Score $< 60\%$ → Not Recommended	PASS
TC-09	Model accuracy $\geq 90\%$	Full test dataset	Classification accuracy ≥ 0.90	PASS
TC-10	Batch screening 5 resumes	5 different resume files	5 separate HR reports generated	PASS